NATURAL LANGUAGE PROCESSING

Enterprise Technical Assessment Report

Scope: repository reverse engineering, architecture review, system audit, and technical due diligence

Assessment date	2026-06-06
Evidence order	Code, config, infrastructure, tests, legacy docs
Primary runtime	FastAPI + in-process model runtime
Primary UI	Angular single-page application behind Nginx
Primary ML toolchain	DVC, HuggingFace, PEFT/LoRA, MLflow, ONNX Runtime

Tech Lead	Le Quang Tuyen
Instructor	Le Thanh Hai
ML Engineer	Duong Binh An
AI Engineer	Do Quoc Trung
Solution Engineer	Duong Hong Quan
Backend Engineer	Pham Duc Long
UI/UX Engineer	Nguyen Le Hong Nhi

Assessment method

The report is derived from repository inspection. Missing code paths are marked explicitly as Implemented, Partially Implemented, Mocked, Deprecated, or Not Implemented. Claims are limited to implementation evidence present in the repository on 2026-06-06.

Contents

Executive Report

The repository implements a sentiment-analysis product with two tightly coupled planes. The online plane is an Angular frontend served by Nginx, a FastAPI backend in `src/main.py`, and an in-process inference engine centered on `src/model/baseline.py`. The offline plane is an ML artifact pipeline defined by `dvc.yaml`, `params.yaml`, `src/data/`, `src/training/`, and `src/scripts/`. The online plane consumes artifacts produced by the offline plane rather than training models at request time.

The strongest implementation evidence is end-to-end completeness. The repository contains real prediction endpoints, real explainability, synchronous CSV batch processing, DVC-driven data and training stages, ONNX export, Prometheus metrics, Grafana provisioning, MLflow integration, and an automated test suite across contracts, data, model, scripts, and training. The weakest areas are enterprise hardening and operational durability. Authentication is absent. Authorization is absent. `/batch_status/{job_id}` is mocked. Evaluation status is process-memory only. The backend is a single process that owns HTTP serving, model loading, SHAP, ABSA, and batch inference.

The system is best classified as a **hybrid architecture: modular monolith runtime plus embedded ML platform**. It is not a microservice system. The serving layer is one FastAPI deployable. The frontend, Prometheus, Grafana, and MLflow are separate containers, but they do not own domain-specific service boundaries. The ML platform is repository-local and script-driven rather than service-oriented.

Final maturity view
Classification: Hybrid Architecture / Modular Monolith with embedded ML pipeline
Current maturity: Advanced MVP
Production readiness: limited internal production or controlled pilot only
Overall technical verdict: implementation-rich system with clear educational and demo value, but incomplete enterprise controls

1 Section 1 - Executive Technical Summary

1.1 System Purpose

The system converts free-form text into structured sentiment outputs for interactive use and CSV batch use. It also demonstrates the full AI application lifecycle: data download, preprocessing, validation, finetuning, evaluation, ONNX export, runtime loading, prediction, explanation, and monitoring.

1.2 Technical Scope

- frontend user experience in `app/sentiment-analysis-chatbot/`
- backend API runtime in `src/main.py`

- model orchestration in `src/model/`
- contracts in `contracts/`
- data, training, evaluation, and export lifecycle in `src/data/`, `src/training/`, `src/scripts/`, `dvc.yaml`, and `params.yaml`
- deployment and observability manifests in `Dockerfile`, `Dockerfile.train`, `docker-compose.yml`, and `infra/`

1.3 Technology Overview

Layer	Technology evidence
Frontend	Angular 17 standalone components, signals, RxJS, Lucide icons, Nginx frontend container
Backend	Python 3.11, FastAPI, Uvicorn, Pydantic models from `contracts/schemas.py`
Inference	Transformers, PEFT, PyTorch, ONNX Runtime, SHAP, zero-shot ABSA
ML lifecycle	DVC, pandas, datasets, scikit-learn, HuggingFace Trainer, MLflow
Observability	Prometheus client, scrape config in `infra/prometheus/prometheus.yml`, Grafana provisioning
Packaging	Docker multi-stage runtime image, dedicated training image, Docker Compose topology

1.4 Core Capability Status

Capability	Status	Implementation evidence
Single prediction	Implemented	`POST /predict` in `src/main.py`; frontend uses `SentimentAnalysisService.predict()`
Explainability	Implemented	`POST /explain` and `BaselineModelInference.get_shap_explanation()`
CSV batch prediction	Implemented	`POST /batch_predict` parses upload, caps to 500 rows, dispatches batch inference via `asyncio.to_thread()`
Durable batch jobs	Mocked	`GET /batch_status/{job_id}` returns a hardcoded completed payload
Background evaluation trigger	Partially Implemented	`/evaluate` starts a background task, but status lives in process memory only
Training pipeline	Implemented	DVC stages plus `src/scripts/run_finetuning.py` and `src/training/`
ONNX export	Implemented	`src/scripts/export_onnx.py` and `src/model/onnx_exporter.py`
Monitoring	Partially Implemented	Prometheus metrics and Grafana provisioning exist; tracing and structured logs do not
Authentication / authorization	Not Implemented	no auth middleware, no dependency, no identity provider, no route protection

1.5 Technical Maturity Assessment

The repository exceeds a toy prototype because it includes a functioning UI, an explicit API contract layer, a concrete inference abstraction, training and export automation, test coverage, and infrastructure manifests. It does not reach enterprise production maturity because trust boundaries, persistent job state, secure secret handling, and workload isolation are missing.

1.6 System Classification

Classification: Hybrid Architecture.

Justification:

- the request-serving backend is one FastAPI monolith with one global model instance
- the frontend is a separate Angular/Nginx container, but not an independently versioned domain service
- monitoring services are support services rather than business services
- the repository embeds a full ML artifact pipeline with DVC and MLflow

2 Section 2 - System Architecture Analysis

2.1 Context Diagram

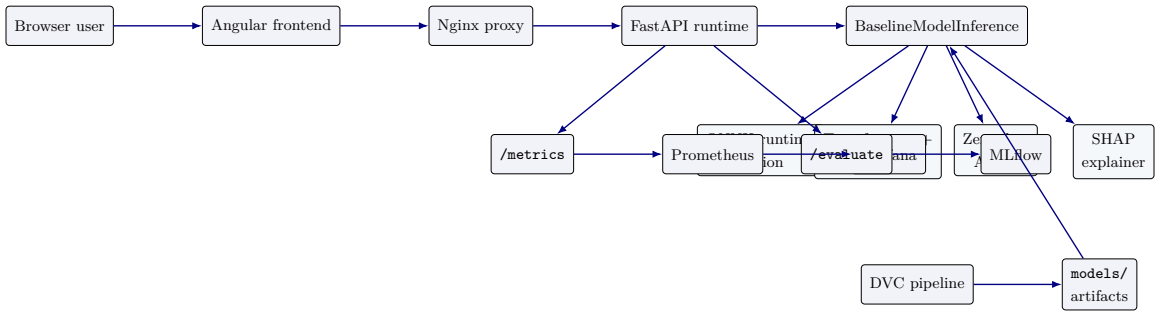

Figure 1: System context and runtime flow

2.2 Trust Boundary Diagram

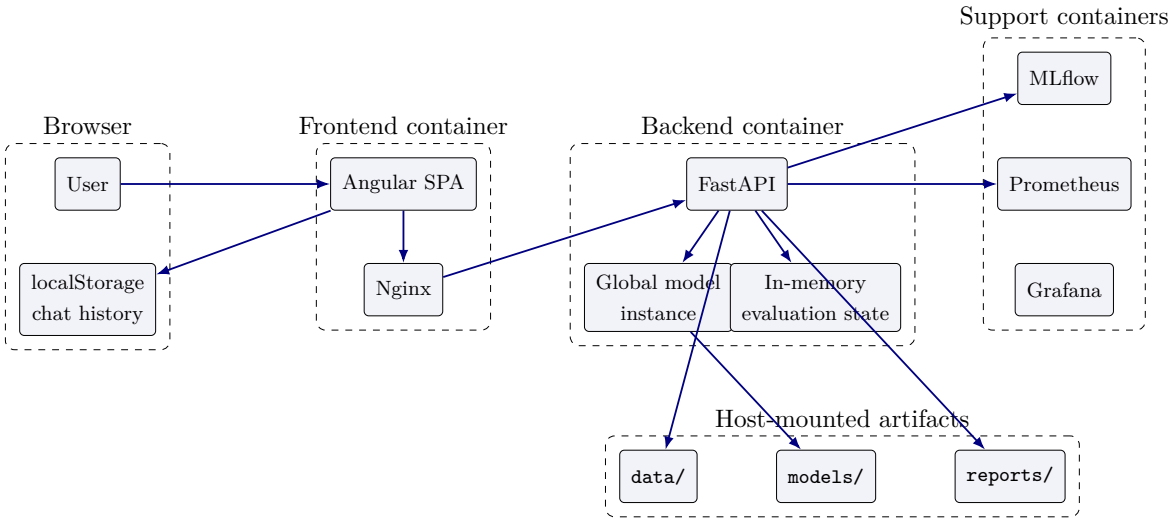

Figure 2: Trust boundaries and state ownership

2.3 Layer Assessment

Layer	Runtime role	Key files	Primary risk
Frontend	browser UI, state, health gating, batch UX	`app.component.ts` `sentiment-analysis.service.ts` batch/message components	stale state shared browser
Backend	HTTP API, validation, metrics, evaluation trigger	`src/main.py` `contracts/schemas.py`	single process public routes schema drift

Layer	Runtime role	Key files	Primary risk
Inference	prediction, explainability, ABSA, sarcasm	`baseline.py` `onnx_inference.py` `config.py`	CPU-heavy ABSA and SHAP
Training	finetuning, evaluation, benchmark, export	`run_finetuning.py` `evaluate_finetuned.py` `benchmark_onnx.py`	file pipeline export drift
Data	ingest, preprocess, validate, eval set prep	`src/data/` `prepare_eval.py` `dvc.yaml`	external data transform correctness
Monitoring	counters, latency, scrape, dashboards	`src/monitoring/metrics.py` `infra/prometheus/` `infra/grafana/`	no tracing no log sink
Deployment	image build, artifact pull, env wiring	`Dockerfile` `Dockerfile.train` `docker-compose.yml`	build secrets provenance scale limits
Security	CORS, secrets, artifact access, trust boundaries	runtime config and default network behavior	no auth wildcard CORS default creds

3 Section 3 - Repository Structure Analysis

3.1 Repository Structure Diagram

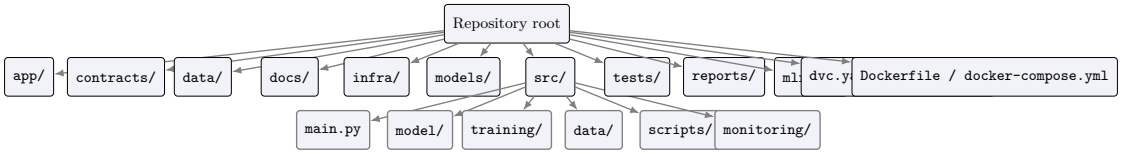

Figure 3: Repository structure and main dependency concentration

3.2 Critical Path and Runtime Relevance

Directory	Criticality	Runtime relevance	Assessment
`src/model/`	Critical	Direct runtime path	most important backend module set; owns model mode selection, ONNX/HF loading, ABSA, SHAP, and batch behavior
`src/main.py`	Critical	Direct runtime path	public API surface and startup lifecycle; breaking changes here are externally visible immediately
`contracts/`	High	Direct runtime path	shared boundary between HTTP layer, tests, and model abstraction; contract drift here is high-risk
`app/`	High	Direct runtime path	defines actual end-user capability set, health gating, batch UX, and local state behavior
`models/`	Critical	Direct runtime dependency	serving depends

3.3 Directory Assessment

Directory	Purpose	Criticality	Dependencies	Modification risk	Complexity	Status
app/	Angular UI and Nginx frontend build	High	backend API and browser APIs	High	Medium	Implemented
contracts/	shared schemas, errors, interface, mock model	High	FastAPI, tests	High	Low	Implemented
data/	raw, processed, eval, and quality artifacts	High	DVC stages and scripts	High	Medium	Implemented
infra/	Prometheus scrape rules and Grafana provisioning	Medium	Compose network and metrics endpoint	Medium	Low	Implemented
models/	LoRA adapters and ONNX artifacts	Critical	training/export scripts and runtime loader	Critical	Medium	Implemented
src/data/	dataset download, preprocess, validate	High	params and raw data	High	Medium	Implemented
src/model/	inference, evaluation, ONNX runtime and export support	Critical	ML libraries and artifacts	Critical	High	Implemented
src/monitoring/	request counters and latency histograms	Medium	FastAPI middleware	Medium	Low	Partially Implemented
src/scripts/	CLI lifecycle tasks	High	training, data, model modules	High	Medium	Implemented
src/training/	trainer, dataset, metrics, task config helpers	High	transformers and sklearn stack	High	Medium	Implemented
tests/	API, contracts, model, data, scripts, training coverage	Medium	pytest and mocks	Medium	Medium	Implemented

3.4 Dependency Graph

```
flowchart LR
  APP --> CONTRACTS
  APP --> API[src/main.py]
```

```
  API --> CONTRACTS
  API --> MODEL[src/model]
  API --> MON[src/monitoring]
  MODEL --> CFG[src/model/config.py]
  MODEL --> ONNX[src/model/onnx_inference.py]
  SCRIPTS[src/scripts] --> TRAIN[src/training]
  SCRIPTS --> DATA[src/data]
  SCRIPTS --> MODEL
  DVC[dvc.yaml] --> SCRIPTS
  DVC --> DATA
  DVC --> TRAIN
  MODEL --> ART[models/]
```

3.5 Module Interaction Diagram

```
flowchart LR
  AppComponent --> SentimentService
  SentimentService --> PredictAPI[/predict]
  SentimentService --> ExplainAPI[/explain]
  SentimentService --> BatchAPI[/batch_predict]
  SentimentService --> HealthAPI[/health]
  PredictAPI --> BaselineModelInference
  ExplainAPI --> BaselineModelInference
  BatchAPI --> BaselineModelInference
  BaselineModelInference --> OnnxInferenceSession
  BaselineModelInference --> HFModel
  BaselineModelInference --> ABSA
  BaselineModelInference --> SHAP
```

4 Section 4 - Execution Flow Analysis

4.1 Workflow Matrix

Workflow	Entry point	Execution path	Key files	Outputs	Status
Startup	FastAPI lifespan	env → `ModelConfig` → `BaselineModelInference` → model load → ABSA preload	`src/main.py`, `src/model/baseline.py`, `src/model/config.py`	global loaded model or startup abort	Implemented
Health check	`GET /health`	dependency injection → `get_model()` → static payload build	`src/main.py`, `contracts/schemas.py`	`HealthResponse`	Implemented
Prediction	`POST /predict`	validate → resolve language → predict → metrics → shape response	`src/main.py`, `baseline.py`, `language_detector.py`	`PredictResponse`	Implemented
Explanation	`POST /explain`	validate → resolve language → SHAP explanation → response	`src/main.py`, `baseline.py`	`ExplainResponse`	Implemented

Workflow	Entry point	Execution path	Key files	Outputs	Status
Batch processing	`POST /batch_predict`	read upload → parse CSV → cap 500 → filter empty → threaded batch inference → row rebuild	`src/main.py`, `baseline.py`	`BatchPredictResponse`	Implemented
Batch status	`GET /batch_status/{id}`	return hardcoded completed state	`src/main.py`	fixed status payload	Mocked
Evaluation	`POST /evaluate`	background task → load params → load processed data → evaluate → log MLflow	`src/main.py`, `src/model/evaluate.py`	JSON status + MLflow run	Partially Implemented
Training	CLI / DVC	download/prep → fine-tune adapter → save outputs	`dvc.yaml`, `run_finetuning.py`, `src/training/`	adapters and local MLflow runs	Implemented
ONNX export	CLI / DVC	load adapter → export FP32 → quantize INT8	`export_onnx.py`, `onnx_exporter.py`	ONNX directories	Implemented
Deployment	`docker compose up`	build images → start support services → backend loads artifacts → frontend proxies	Dockerfiles and Compose	multi-container local stack	Implemented

4.2 Startup Sequence

```
sequenceDiagram
  participant Uvicorn
  participant FastAPI
  participant Lifespan
  participant Config as ModelConfig
  participant Model as BaselineModelInference
  participant Artifacts as models/
  Uvicorn->>FastAPI: start app
  FastAPI->>Lifespan: enter lifespan()
  Lifespan->>Config: read MODEL_MODE and build config
  Lifespan->>Model: instantiate global model
  Model->>Artifacts: load ONNX or HF artifacts
  Model->>Model: preload ABSA pipeline
  alt load success
    Lifespan-->>FastAPI: app ready
  else load failure
    Model-->>Lifespan: raise ModelError
    Lifespan-->>FastAPI: startup abort
  end
```

4.3 Prediction Sequence

```
sequenceDiagram
```

```
  participant User
  participant FE as Angular Service
  participant API as FastAPI /predict
  participant Lang as LanguageDetector
  participant Model as BaselineModelInference
  participant Metrics as Prometheus Histograms
  User->>FE: submit text
  FE->>API: POST /predict
  API->>Lang: resolve_request_language()
  API->>Model: predict_single(text, lang)
  Model->>Model: _predict_probabilities()
  Model->>Model: _extract_aspects()
  Model->>Model: _predict_sarcasm_flag()
  API->>Metrics: observe inference latency
  API-->>FE: PredictResponse
  FE-->>User: render sentiment, aspects, sarcasm, latency
```

4.4 Batch Sequence

```
sequenceDiagram
  participant User
  participant FE as BatchUploadComponent
  participant API as FastAPI /batch_predict
  participant Pandas
  participant Worker as asyncio.to_thread
  participant Model as BaselineModelInference
  User->>FE: upload CSV
  FE->>API: multipart POST /batch_predict
  API->>Pandas: read_csv(bytes)
  API->>API: validate text column and cap 500 rows
  API->>Worker: run blocking batch inference
  Worker->>Model: predict_batch(valid_texts, skip_absa=True)
  Model-->>Worker: list of PredictionResult
  API->>API: rebuild per-row results and errors
  API-->>FE: BatchPredictResponse
  FE-->>User: render summary and exportable results
```

4.5 Evaluation Sequence

```
sequenceDiagram
  participant Caller
  participant API as FastAPI /evaluate
  participant Task as BackgroundTask
  participant Params as params.yaml
  participant Data as data/processed/sentences.csv
  participant Eval as src/model/evaluate.py
  participant MLflow
  Caller->>API: POST /evaluate
  API->>API: check _evaluate_state.running
  API->>Task: add _do_evaluate
  API-->>Caller: started
  Task->>Params: load params
```

```
  Task->>Data: read processed data
  Task->>Eval: evaluate_on_dataset()
  Task->>MLflow: log metrics
  Task->>API: update in-memory status
```

5 Section 5 - Frontend Analysis

5.1 Architecture

The frontend is an Angular standalone-component application. `AppComponent` is the shell. `SentimentAnalysisService` owns API calls and chat history state. Batch processing is isolated in `BatchUploadComponent`. Explanation interaction is coordinated by `MessageBubbleComponent`, while health polling happens in `AppComponent.ngOnInit()`.

5.2 Component and State Model

Component / service	Role	Evidence-backed behavior
`AppComponent`	application shell	polls `/api/health` every 3 seconds until success, controls modal visibility, theme, mobile sidebar, and loading overlay
`SentimentAnalysisService`	state and API service	stores messages in an Angular signal, persists chat history to `localStorage`, calls `/api/predict`, `/api/explain`, `/api/batch_predict`, and `/api/health`
`BatchUploadComponent`	batch workflow UI	validates CSV extension on selection, sends multipart request, renders summary cards and detailed table, exports client-side CSV
`MessageBubbleComponent`	result rendering	toggles explanation fetch for prior bot messages and displays returned SHAP payload
`ThemeService`	UI preference	toggles dark mode state locally in frontend only

5.3 Frontend Data Flow

```
flowchart LR
  Input[ChatInput or BatchUpload] --> Service[SentimentAnalysisService]
  Service --> Health[/api/health]
  Service --> Predict[/api/predict]
  Service --> Explain[/api/explain]
  Service --> Batch[/api/batch_predict]
  Service --> Signal[message signal]
  Signal --> UI[AppComponent and child components]
  Signal --> Storage[localStorage]
```

5.4 API Integration

- `checkHealth()` calls `/api/health` with no local schema wrapper.
- `predict()` posts only `{text}`; the current UI does not send `lang` even though the backend schema supports it.
- `callExplainApi()` posts only `{text}` and attaches results to the existing bot message.
- `batchPredict()` submits a multipart `FormData` payload containing one file field named `file`.

5.5 UI Interaction Flows

- startup flow: render loading overlay, poll health, then inject a welcome message
- single prediction flow: append user message, append loading bot message, replace loading state when response arrives
- explainability flow: find the nearest preceding user message, call `/api/explain`, attach SHAP output to the existing bot message
- batch flow: open modal, select CSV, run batch, render summary cards and export results CSV locally

5.6 Strengths, Weaknesses, and Risks

Dimension	Assessment
Strengths	clean service boundary, browser-only persistence is simple, batch UX is concrete, health-gated startup improves user feedback
Weaknesses	no server-persisted session state, no retry policy beyond basic error messages, health contract typed as `any`, no upload progress indicator
Risks	localStorage can become stale or malformed, frontend readiness can be optimistic when chat history exists, no auth-aware UX because auth does not exist
Maintainability concerns	large inline templates in components and business logic mixed with presentation increase change risk

6 Section 6 - Backend Analysis

6.1 FastAPI Architecture

The backend is a single FastAPI application with a global model singleton and request middleware. The app lifecycle is startup-heavy because model loading and ABSA preload happen in the lifespan function. There is no service layer split separate from route handlers; routing, response shaping, and metrics observation remain in `src/main.py`.

6.2 Routing and Dependency Injection

- dependency: `get_model()` returns the global `ml_model` or raises HTTP 503
- helper: `resolve_request_language()` returns explicit `lang` if provided, else uses `LanguageDetector`
- middleware: CORS plus Prometheus middleware from `src/monitoring/metrics.py`
- exception handlers: `UnsupportedLanguageError` mapped to HTTP 400 and `ModelError` mapped to HTTP 500 as `text/plain`

6.3 Validation and Contracts

Validation is primarily schema-driven in `contracts/schemas.py`. `PredictRequest` enforces text length 1 to 2000. `ExplainResponse` validates that `tokens` and `shap_values` lengths match. Batch CSV validation is route-local and checks parseability and presence of a `text` column.

6.4 Backend Control Flow

```
flowchart LR
  Request --> CORS
  CORS --> Monitor[monitor_middleware]
  Monitor --> Route[src/main.py routes]
  Route --> Dependency[get_model()]
  Route --> Lang[resolve_request_language]
  Route --> Model[BaselineModelInference]
  Route --> Schema[Pydantic response models]
  Schema --> Response
```

6.5 Background Tasks and Long-running Work

- batch inference is not a FastAPI background task; it is executed with `asyncio.to_thread()` inside the request scope
- evaluation uses `BackgroundTasks.add_task()` and stores state in module-global `_evaluate_state`
- there is no queue, no worker process, and no durable state store

6.6 Metrics Collection

Middleware records request count and request latency for all paths. The prediction route separately observes model inference latency by normalized language label. There is no metric for batch row count, SHAP latency, ABSA latency, startup duration, or evaluation duration.

7 Section 7 - AI Inference Analysis

7.1 Model Loading Strategy

Mode	Loader path	Behavior
`onnx` / `onnx_int8`	`BaselineModelInference._load_model()` → `OnnxInferenceSession`	loads sentiment ONNX session and attempts to load matching sarcasm ONNX session by path substitution
`finetuned`	tokenizer + base HF model + PEFT sentiment and sarcasm adapters	keeps one model object and switches adapters for sentiment versus sarcasm prediction
`baseline`	tokenizer + base HuggingFace model	no fine-tuned adapter, no sarcasm model, no ONNX

7.2 Inference Pipeline

```
flowchart LR
  Text --> LangGuard[_check_language]
  LangGuard --> Sentiment[_predict_probabilities]
  Sentiment --> SentimentLabel[label_map argmax]
  SentimentLabel --> ABSA[_extract_aspects]
  SentimentLabel --> Sarcasm[_predict_sarcasm_flag]
  SentimentLabel --> Response[PredictionResult]
  Text --> SHAP[get_shap_explanation]
```

7.3 Stage Analysis

Stage	Input	Transformation	Output	Cost profile	Failure modes
Language guard	`text`, optional `lang`	explicit pass-through or detector call	normalized language code	low CPU	unsupported language later raises 400
Sentiment scoring	text or batch chunk	ONNX or HF forward pass, softmax, argmax mapping	class probabilities and label	primary runtime cost	artifact load errors, runtime inference errors
Sarcasm scoring	one text	ONNX sarcasm session or fine-tuned sarcasm adapter	boolean flag	sequential per text	missing sarcasm model degrades to false
ABSA	one text	zero-shot pipeline over configured aspect categories and threshold	list of `AspectSentiment`	expensive CPU path	pipeline load failure at startup or per-call warning with empty result
SHAP	one text	build prediction function, build SHAP explainer, compute token contributions	`SHAPResult`	highest per-request cost	tokenizer/pipeline/SHAP runtime errors
Batch prediction	list of texts	chunked inference, optional ABSA skip and sarcasm skip	list of `PredictionResult`	scales with chunk count and per-row post-processing	invalid batch size, runtime inference failure, iterator mismatch risk if logic changes

7.4 Inference Optimization Status

- implemented: ONNX runtime mode, INT8 export path, batch chunking, language-label normalization for metrics, ABSA preload to shift cost to startup
- partially implemented: batch route skips ABSA to control latency, but sarcasm remains per-row unless explicitly disabled

- not implemented: model pool isolation, request-level timeout control, SHAP caching, ABSA caching, concurrency throttling

8 Section 8 - ML Pipeline Analysis

8.1 DVC Pipeline Graph

```
flowchart TD
  download --> preprocess
  preprocess --> validate
  preprocess --> evaluate_baseline
  download_sarcasm --> finetune_sarcasm
  download_sentiment --> prepare_eval
  download_sentiment --> finetune_sentiment
  finetune_sarcasm --> evaluate_finetuned
  finetune_sentiment --> evaluate_finetuned
  finetune_sarcasm --> export_onnx_sarcasm
  finetune_sentiment --> export_onnx_sentiment
  export_onnx_sentiment --> benchmark_onnx
```

8.2 Pipeline Stage Analysis

Stage	Purpose	Inputs	Outputs / artifacts	Dependencies	Status
`download`	fetch SemEval raw data	`src/data/downloader.py`, external XML files	`data/raw/sentences.csv`, `data/raw/aspects.csv`	DVC, downloader script	Implemented
`preprocess`	transform raw data into processed data	raw CSVs, transforms, params	`data/processed/`	`src/data/pipeline.py`	Implemented
`validate`	quality checks on processed data	processed data and validation params	`data/reports/quality_report.json`	validation module	Implemented
`evaluate_baseline`	baseline evaluation with MLflow params	processed sentences and model code	`data/reports/baseline_metrics.json`	evaluation module	Implemented
`download_sarcasm` / `download_sentiment`	fetch task-specific training CSVs	downloader script and params	task raw CSVs	downloader module	Implemented
`prepare_eval`	build evaluation datasets from raw sentiment splits	raw sentiment CSVs and evaluation params	`data/eval/`	`src/scripts/prepare_eval.py`	Implemented
`finetune_*`	produce task adapters	task data, training helpers, params	`models/adapters/*`	`run_finetuning.py`	Implemented

Stage	Purpose	Inputs	Outputs / artifacts	Dependencies	Status
`evaluate_finetuned`	evaluate fine-tuned artifacts and fairness	adapters and eval sets	metrics JSON reports in `reports/`	eval script and model modules	Implemented
`export_onnx_*`	create serving artifacts	adapters and ONNX exporter	FP32 and INT8 directories	export script	Implemented
`benchmark_onnx`	benchmark runtime artifact performance	ONNX artifacts and benchmark script	`reports/onnx_benchmark.json`	benchmark script	Implemented

8.3 ML Lifecycle Observations

- the pipeline is artifact-oriented rather than service-oriented
- DVC is the orchestration source of truth for offline sequencing
- MLflow is used for experiment logging, but there is no model registry promotion workflow in code
- runtime artifacts and training outputs are colocated under `models/`, which simplifies local use but weakens environment isolation

9 Section 9 - Data Flow Analysis

9.1 End-to-End Data Lineage

```
flowchart LR
  Raw[Raw XML and downloaded CSV] --> Prep[Preprocess and validation]
  Prep --> Processed[data/processed]
  Processed --> Train[Finetuning]
  Processed --> EvalBase[Baseline evaluation]
  Train --> Adapters[models/adapters]
  Adapters --> Export[ONNX export]
  Export --> OnnxArtifacts[models/onnx]
  OnnxArtifacts --> Runtime[BaselineModelInference]
  Runtime --> Prediction[Predict and Explain APIs]
```

9.2 Artifact Inventory

Artifact	Type	Produced by	Consumed by	Location
SemEval raw extracts	raw tabular data	`download` stage	`preprocess`	`data/raw/sentences.csv`, `data/raw/aspects.csv`
Processed training data	processed dataset	`preprocess`	validation, baseline evaluation, downstream analysis	`data/processed/`

Artifact	Type	Produced by	Consumed by	Location
Quality report	metrics JSON	`validate`	human review / CI evidence	`data/reports/quality_report.json`
Task raw datasets	raw CSVs	task-specific download stages	finetuning, eval prep	`data/raw/sarcasm.csv`, `data/raw/sentiment_*.csv`
Evaluation sets	evaluation CSVs	`prepare_eval`	finetuned evaluation	`data/eval/`
LoRA adapters	model artifacts	`finetune_*`	ONNX export, finetuned runtime mode	`models/adapters/`
ONNX sentiment / sarcasm models	runtime model artifacts	`export_onnx_*`	ONNX runtime serving	`models/onnx/`
Benchmark and metrics reports	JSON reports	eval and benchmark scripts	human review / docs	`reports/`, `data/reports/`

9.3 Transformation Logic

- preprocessing transforms external raw data into normalized sentence-level and aspect-level forms
- validation enforces data quality and writes a non-cached report
- finetuning converts labeled task datasets into adapters
- export converts adapters into ONNX artifacts for low-latency serving
- runtime never reads training code directly; it reads exported artifacts and static config

10 Section 10 - API Analysis

10.1 Endpoint Inventory

Method	Path	Purpose	Request / response	Key validation	Auth
GET	`/health`	expose model readiness and supported languages	no request; `HealthResponse`	model dependency must exist	None
POST	`/predict`	single-text prediction	`PredictRequest` → `PredictResponse`	text length 1–2000, language support enforced by model	None
POST	`/explain`	token-level SHAP explanation	`ExplainRequest` → `ExplainResponse`	schema plus token/value length equality on response	None

Method	Path	Purpose	Request / response	Key validation	Auth
POST	`/batch_predict`	synchronous CSV batch prediction	multipart file → `BatchPredictResponse`	CSV parse, `text` column present, first 500 rows only	None
GET	`/batch_status/{job_id}`	report batch job status	path param → `BatchStatusResponse`	none meaningful; returns fixed payload	None
POST	`/evaluate`	trigger async evaluation	no formal body; JSON status object	only one in-flight run allowed	None
GET	`/evaluate/status`	inspect in-memory evaluation state	no request; generic JSON	none	None
GET	`/metrics`	expose Prometheus metrics	no request; Prometheus plain-text	none	None

10.2 Request and Failure Behavior

Endpoint	Failure behavior
`/health`	returns 503 indirectly if `get_model()` raises because startup failed or is incomplete
`/predict`	unsupported language maps to 400, generic model errors map to 500 `text/plain`
`/explain`	same model-related exception mapping as `/predict`; SHAP errors bubble through `ModelError` if wrapped or generic 500 otherwise
`/batch_predict`	invalid CSV and missing `text` column return 400; inference failures return 500
`/evaluate`	returns 409 if evaluation already running; background task records last error in memory only
`/metrics`	no dedicated auth or throttling, so scrape endpoint is publicly reachable wherever the API is reachable

11 Section 11 - Deployment Analysis

11.1 Compose Topology

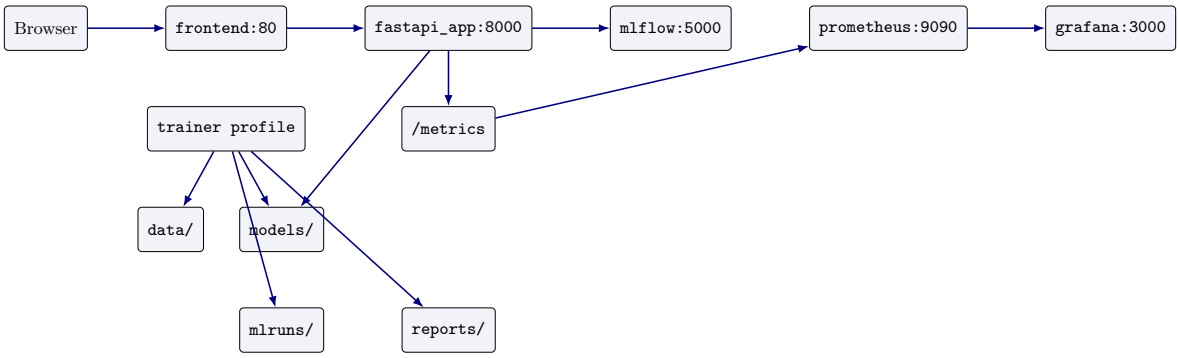

Figure 4: Deployment topology in Docker Compose

11.2 Service Inventory

Service	Role	Ports	Notable config
`fastapi_app`	runtime API and model host	`8000:8000`	DVC-pulled artifacts in image build, CPU and memory limits, MLflow tracking URI
`frontend`	Angular UI over Nginx	`80:80`	depends on backend, proxies API traffic
`prometheus`	scrape metrics	`9091:9090`	scrape config and alert rules mounted from `infra/prometheus/`
`grafana`	dashboards	`3000:3000`	admin password set to `admin`, provisioning mounted from repo
`mlflow`	experiment tracking server	`5005:5000`	local server mode, no auth configuration
`trainer`	optional offline training container	profile only	host-mounted `data/`, `models/`, `mlruns/`, `reports/`

11.3 Dockerfile Findings

- runtime image installs DVC and pulls selected ONNX artifacts and processed sentences during build
- build requires DagsHub credentials through `DAGSHUB_USERNAME` and `DAGSHUB_TOKEN`
- runtime image intentionally pulls sentiment and sarcasm ONNX artifacts rather than training inside the serving container
- training has a separate `Dockerfile.train` that excludes ONNX runtime and is used only through the Compose `trainer` profile

11.4 Environment Variables and Startup Order

Variable	Consumer	Purpose
`MODEL_MODE`	FastAPI startup	selects ONNX, ONNX INT8, fine-tuned, or baseline mode
`MLFLOW_TRACKING_URI`	backend and trainer	selects MLflow destination
`OMP_NUM_THREADS`, `MKL_NUM_THREADS`	backend and trainer	adjusts CPU thread usage
`DAGSHUB_USERNAME`, `DAGSHUB_TOKEN`	runtime image build	required for DVC artifact pull during image build
`GF_SECURITY_ADMIN_PASSWORD`	Grafana	sets admin password, currently `admin` in compose

12 Section 12 - Observability Analysis

12.1 Logging and Metrics

Metrics are implemented. Logging is minimal and mostly implicit through Python logging in model code plus framework defaults. There is no structured JSON logging, no correlation ID, and no log sink configuration in Compose.

12.2 Metrics Inventory

Metric	Type	Evidence-backed meaning
`api_requests_total`	Counter	counts requests by method, endpoint, and HTTP status in middleware
`api_request_latency_seconds`	Histogram	measures whole-request latency by method and endpoint
`model_inference_latency_seconds`	Histogram	records latency for `/predict` model inference only, labeled by normalized language

12.3 Telemetry Flow

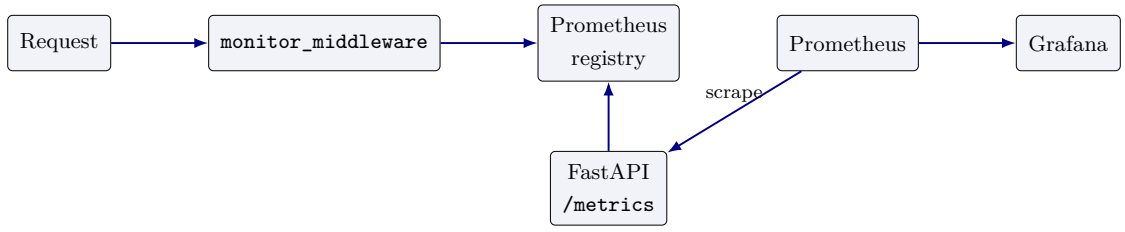

Figure 5: Telemetry collection and scrape path

12.4 Monitoring Coverage and Gaps

- covered: endpoint volume, endpoint latency, prediction inference latency

- partially covered: alerting files exist, but alert operational quality is not proven by tests here
- not covered: startup duration, model load failures, SHAP latency, ABSA latency, batch row counts, queue depth, memory, CPU, error taxonomy
- not implemented: tracing, log aggregation, SLO reporting, anomaly detection

13 Section 13 - Security Analysis

13.1 Evidence-based Findings

Severity	Finding	Evidence	Impact	Recommendation
Critical	no authentication or authorization on any route	`src/main.py` exposes all endpoints without auth dependencies	any reachable client can call prediction, explanation, evaluation, and metrics	add API auth at gateway or app layer before external exposure
High	wildcard CORS allows all origins with credentials enabled	`CORSMiddleware` `allow_origins=["*"]` and `allow_credentials=True`	broad browser attack surface and poor boundary hygiene	restrict allowed origins and disable credential support unless required
High	Grafana default admin password is committed in compose	`GF_SECURITY_ADMIN_PASSWORD=admin` in `docker-compose.yml`	trivial compromise in exposed environments	externalize secret and rotate default admin credentials
High	build-time artifact pull requires secrets in image build context	`Dockerfile` uses DVC pull with DagsHub credentials	secret mishandling risk in local and CI builds	move to secure build secret mechanism or pre-provision artifacts
Medium	admin-style evaluation trigger is public	`POST /evaluate` has no access control	untrusted users can force expensive background evaluation	protect the route or remove from serving deployment
Medium	metrics endpoint is public	`GET /metrics` has no auth	leaks operational metadata to any reachable caller	network-restrict or gateway-protect metrics
Medium	batch upload trusts file extension at frontend and content parse at backend	frontend checks extension; backend parses arbitrary CSV bytes	malformed or oversized uploads can cause resource pressure	add size limits, MIME checks, and server-side quotas
Low	localStorage retains prior chat content	`SentimentAnalysisService.savePersistedState()`	shared browsers may expose prior user text	add explicit privacy notice or opt-out setting

14 Section 14 - Performance Analysis

14.1 Observed Architecture Constraints

The repository includes a benchmark stage for ONNX but the report cannot claim general production timings without executing the workloads. What is implementation-evident is the performance shape:

- startup is expensive because the app loads the primary model and preloads the ABSA pipeline in lifespan
- `/predict` performs one sentiment inference plus ABSA plus optional sarcasm logic in one request
- `/explain` is heavier than `/predict` because it constructs SHAP explanations per request
- `/batch_predict` skips ABSA to reduce cost, but still performs full sentiment inference and row reconstruction synchronously
- the backend remains CPU-sensitive because the compose file explicitly increases thread counts and CPU limits

14.2 Performance Risk Matrix

Area	Risk level	Evidence-backed reason
Startup time	High	model load and ABSA preload both occur before service readiness
Single prediction latency	Medium	one request can include language resolution, sentiment, ABSA, and sarcasm
Explain latency	High	SHAP explanation is generated on demand and not cached
Batch throughput	Medium	chunked inference exists, but request remains synchronous and process-local
Memory consumption	High	backend holds global model state and may also load ABSA and sarcasm ONNX sessions
Horizontal scalability	High	singleton in-process model and local memory state complicate concurrent scale-out semantics

15 Section 15 - Failure Mode Analysis

15.1 Failure Tree Overview

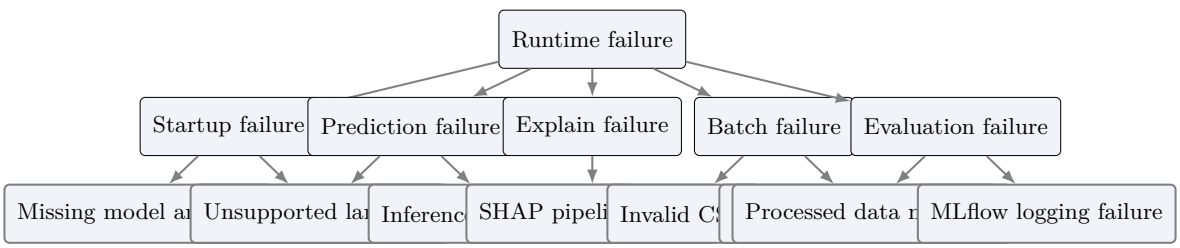

Figure 6: Fault-tree view of the dominant runtime failure modes

15.2 Failure Mode Table

Workflow	Symptoms	Probable causes	Affected components	Recovery path
Startup	API never becomes healthy	missing artifacts, bad model mode, ABSA preload error	backend, frontend readiness, monitoring scrape target	fix artifact/config issue and restart container
Prediction	400 or 500 response, frontend error bubble	unsupported language, model runtime exception, unloaded model	backend route, model layer, frontend chat flow	inspect model mode and artifacts; validate input language
Explanation	slow or failed explanation request	SHAP initialization, tokenizer/pipeline issues, resource pressure	explain route and message bubble UX	retry after model stabilization; consider disabling SHAP in weak environments
Batch processing	modal shows API error, no results returned	invalid CSV, missing text column, threaded inference failure	batch route and batch modal	fix input file or investigate model/runtime exception
Evaluation	status remains failed or last error populated	processed data missing, MLflow issue, evaluation exception	background task and MLflow integration	inspect `data/processed/sentences.csv` and MLflow URI
Training	adapter not produced	raw data missing, training config issue, trainer failure	DVC stages and scripts	rerun required upstream stages and verify params
Deployment	service starts partially	missing build secrets, DVC pull failure, compose misconfig	container stack	rebuild with valid secrets or pre-fetched artifacts
Monitoring	dashboards empty or stale	scrape target unavailable, misconfigured data source	Prometheus and Grafana	verify `/metrics` and provisioned configs

16 Section 16 - Technical Debt Analysis

16.1 Debt Register

Debt type	Severity	Evidence
Architectural debt	High	serving backend mixes HTTP, orchestration, and long-running evaluation control in one file and one process
Operational debt	High	evaluation state is in memory only, batch status is mocked, no durable job store
Security debt	Critical	no auth, wildcard CORS, default Grafana admin password, public admin-style routes
ML debt	Medium	artifact lifecycle and runtime mode selection are local-file based rather than registry-governed
Testing debt	Medium	tests exist broadly, but no evidence here of end-to-end container smoke tests or auth-related tests because auth is absent
Documentation debt	Medium	older docs still describe the system as a microservice architecture, while code shows a modular monolith
Frontend debt	Medium	large inline templates and UI logic concentration in `AppComponent` and batch component

17 Section 17 - Scalability Analysis

17.1 Application Scalability

The backend can scale only within the limits of in-process model serving. Horizontal scale is possible at the container level, but stateful assumptions remain weakly defined because evaluation state is local to one process and there is no shared queue or job store.

17.2 Model Scalability

- sentiment scoring scales better in ONNX mode than SHAP and ABSA
- sarcasm scoring adds per-text overhead
- SHAP does not scale well for high request concurrency in the current design

17.3 Operational Scalability

- local Compose is sufficient for development and demos
- there is no orchestration manifest for Kubernetes or rolling replacement
- there is no externalized queue, cache, or database for cross-instance coordination

17.4 Primary Bottlenecks

- startup model load time
- SHAP request cost
- ABSA request cost

- synchronous batch route
- single backend process owning mixed workloads

18 Section 18 - Architectural Assessment

18.1 Scorecard

Dimension	Score / 10	Justification
Architecture quality	7	coherent modular monolith with explicit online and offline planes, but service boundaries remain coarse
Maintainability	6	contracts and tests help, but backend and frontend central files are still too concentrated
Reliability	5	core inference path works, but startup heaviness, in-memory state, and mocked batch status reduce resilience
Security	3	no auth, permissive CORS, weak defaults, and public admin-style routes
Performance	6	ONNX path and batch chunking help, but SHAP and ABSA are expensive and colocated with serving
Observability	5	metrics and dashboards exist, but visibility remains shallow and logging is minimal
Extensibility	7	model interface, DVC stages, and script layout support extension reasonably well
Operational readiness	5	Compose and Dockerfiles exist, but rollout, secret handling, and durable job control are weak
Testing maturity	7	broad unit and module test surface across contracts, data, model, scripts, and training
ML lifecycle maturity	7	clear artifact path with DVC, MLflow, finetuning, export, and benchmarking

19 Section 19 - Recommendations

19.1 Immediate Improvements

Problem	Evidence	Priority	Complexity	Expected outcome
Unauthenticated public surface	all routes in `src/main.py` are public	Critical	Medium	basic production boundary control
Mocked batch status	`/batch_status` returns fixed payload	High	Medium	truthful batch semantics or explicit endpoint removal
In-memory evaluation state	`_evaluate_state` is process-local	High	Medium	durable multi-run observability
Weak dashboard security	Grafana password is `admin`	High	Low	reduce trivial support-service compromise

Problem	Evidence	Priority	Complexity	Expected outcome
Permissive CORS	wildcard origin with credentials	High	Low	stronger browser boundary hygiene

19.2 Short- to Long-term Roadmap

- Short term: move evaluation and batch jobs to durable job records; add upload size limits and route protection; emit richer metrics for batch, startup, SHAP, and evaluation.
- Medium term: split long-running and heavy analysis workloads from the request-serving API; introduce a proper config and secret strategy per environment.
- Long term: adopt model registry governance, environment-specific artifact promotion, and workload isolation for explanation and offline evaluation.

20 Section 20 - Final Technical Conclusion

The repository is a technically meaningful full-stack AI system rather than a partial demo. It contains a real user interface, a real FastAPI contract surface, a functional inference abstraction, a reproducible artifact pipeline, exportable ONNX serving assets, monitoring integration, and broad tests. Those are genuine strengths.

The decisive weaknesses are not missing architecture diagrams or missing concepts; they are missing production controls. The current implementation does not protect endpoints, does not persist long-running job state, uses a mocked batch-status endpoint, colocates heavy analysis with request serving, and exposes support services with weak defaults.

Final verdict: **strong implementation-first assessment target, suitable for learning, demonstration, and controlled internal use; not yet suitable for open production exposure without security, durability, and workload-isolation remediation.**

21 Section 21 - Dependency Analysis

21.1 Package Dependency Graph

The runtime package surface is concentrated in `requirements.txt` and is exercised by `src/main.py`, `src/model/baseline.py`, `src/model/onnx_exporter.py`, `src/scripts/run_finetuning.py`, and the frontend build in `app/sentiment-analysis-chatbot/package.json`. The important coupling is not package count; it is that the serving path depends on `fastapi`, `pydantic`, `pandas`, `torch`, `transformers`, `peft`, `onnxruntime`, `shap`, `prometheus-client`, and `uvicorn` all at once.

```
flowchart LR
  main[src/main.py] --> fastapi[FastAPI / Uvicorn]
  main --> schemas[contracts/schemas.py]
  main --> baseline[src/model/baseline.py]
```

```
  baseline --> transformers[transformers]
  baseline --> torch[torch]
  baseline --> onnx[onnxruntime]
  baseline --> shap[shap]
  baseline --> peft[peft]
  baseline --> device[src/model/device.py]
  main --> metrics[src/monitoring/metrics.py]
  train[src/scripts/run_finetuning.py] --> datasets[datasets / pandas / sklearn]
  train --> trainer[transformers.Trainer]
  export[src/model/onnx_exporter.py] --> optimum[optimum.onnxruntime]
  export --> peft
  frontend[app/sentiment-analysis-chatbot] --> angular[Angular / RxJS / HttpClient]
```

21.2 Module Dependency Graph

```
flowchart TD
  A[src.main] --> B[contracts.schemas]
  A --> C[contracts.model_interface]
  A --> D[src.model.baseline]
  D --> E[src.model.config]
  D --> F[src.model.onnx_inference]
  D --> G[src.model.language_detector]
  D --> H[contracts.errors]
  D --> I[src.monitoring.metrics]
  D --> J[src.model.device]
  K[src.data.pipeline] --> L[src.data.transforms]
  K --> M[src.data.utils]
  N[src.scripts.run_finetuning] --> O[src.training]
  N --> P[src.training.task_configs]
  N --> Q[src.training.dataset_builder]
  N --> R[src.training.class_weights]
  S[app sentiment-analysis-chatbot] --> T[SentimentAnalysisService]
  T --> A
```

21.3 Runtime Dependency Graph

- backend runtime requires artifacts from `models/onnx/` or HuggingFace downloads handled by `src/model/download_models.py`
- health and prediction depend on `BaselineModelInference.is_loaded` and `get_model()` in `src/main.py`
- batch prediction depends on `pandas.read_csv()` plus `asyncio.to_thread()` because the batch path is intentionally threaded
- training depends on raw CSVs under `data/raw/` and task settings in `src/training/task_configs.py`
- monitoring depends on `monitor_middleware()` and the Prometheus scrape path in `infra/prometheus/prometheus.yml`

21.4 Container Dependency Graph

```
flowchart LR
  FE[frontend container] --> API[fastapi_app container]
  API --> MLFLOW[mlflow container]
  PROM[prometheus container] --> API
  GRAF[grafana container] --> PROM
  TRAIN[trainer profile container] --> MLFLOW
  TRAIN --> DATA[data/ volume]
  TRAIN --> MODELS[models/ volume]
  API --> MODELS
```

21.5 Model Dependency Graph

The model dependency chain is explicit: `ModelConfig` determines mode and artifact paths; `BaselineModelInference` loads either ONNX or HuggingFace/PEFT backends; `OnnxInferenceSession` wraps ONNX Runtime sessions; `predict_single()`, `predict_batch()`, `_extract_aspects()`, `_predict_sarcasm_flag()`, and `get_shap_explanation()` are the operational methods.

21.6 Coupling, upgrade risk, and single points of failure

Coupling area	Evidence	Risk	Severity	Impact
Model/runtime	`src/main.py` + `src/model/baseline.py`	startup and request path both depend on artifact availability	High	backend outage if artifacts or mode are wrong
Frontend/backend contract	`SentimentAnalysisService` + `contracts/schemas.py`	request/response drift breaks UI rendering	High	broken chat and batch UX
ONNX/PEFT export chain	`src/model/onnx_exporter.py` + `dvc.yaml`	export changes ripple into deployment paths	Medium	stale or missing serving artifacts
Monitoring	`src/monitoring/metrics.py` + `infra/prometheus/`	shallow metrics can miss degradation	Medium	slower incident detection
Trainer and MLflow	`src/scripts/run_finetuning.py` + `docker-compose.yml`	experiment tracking and model output share local volumes	Medium	local state corruption / ambiguity

22 Section 22 - Maintainability Analysis

22.1 Code organization

The repository is organized as a two-plane system: online serving under `src/main.py`, `src/model/`, `contracts/`, and `app/`; offline artifact production under `src/data/`,

`src/training/`, `src/scripts/`, `dvc.yaml`, and `params.yaml`. This is coherent, but it concentrates important behavior in `src/main.py` and `src/model/baseline.py`.

22.2 Complexity assessment

- `BaselineModelInference` is the highest-complexity class because it multiplexes baseline HF, finetuned PEFT, ONNX, ABSA, sarcasm, and SHAP behavior
- `src/main.py` is the highest-complexity route file because it owns startup, routing, validation, batch, evaluation, and metrics wiring
- `src/scripts/run_finetuning.py` is the highest-complexity training entrypoint because it manages data loading, balancing, tokenization, PEFT, trainer construction, and MLflow integration

22.3 Coupling assessment

The tightest couplings are `src/main.py` ↔ `src/model/baseline.py`, `app/.../sentiment-analysis.service.ts` ↔ `src/main.py`, and `src/scripts/export_onnx.py` ↔ `src/model/onnx_exporter.py`. These couplings are intentional, but they mean that interface changes require coordinated edits across backend, frontend, tests, and docs.

22.4 Separation of concerns

Separation is good at the repository boundary, weaker inside the runtime boundary. The code separates contracts, data, training, and model modules well, but the runtime service still combines request handling, inference orchestration, batch processing, and evaluation triggers in one deployable.

22.5 Testability

Testability is materially better than the average demo repository because there are tests for contracts, data, training, model, scripts, API behavior, and performance. The main testability limitations are global model initialization in `lifespan()` and the dependence on local artifact paths.

22.6 Extensibility

The strongest extension points are `ModelInference`, `TaskConfig`, and the DVC stage graph. The weakest extension points are the hardcoded route semantics in `src/main.py` and the global, process-local evaluation state.

22.7 God classes and high-risk modules

Module/class	Why it is high risk	Refactor candidate	Priority
`src.model.baseline.BaselineModelInference`	aspect selection, ABSA, sarcasm, SHAP, and batching	split explainability and auxiliary models behind adapter interfaces	High
`src.main`	mixes API, startup, metrics, batch, evaluation orchestration	extract route services and background job manager	High
`src.scripts.run_finetuning`	owns end-to-end training flow	extract dataset, trainer, and logging orchestration helpers	Medium
`app/.../SentimentAnalysisService`	explain, batch, and history persistence	split batch and explain services from chat state	Medium

23 Section 23 - Technical Ownership & Risk Analysis

23.1 Folder ownership matrix

Folder	Primary owner role	Secondary owner role	Operational risk	Bus factor
`app/`	UI/UX Engineer	Solution Engineer	frontend/API drift	Medium
`contracts/`	Backend Engineer	AI Engineer	schema drift	Medium
`src/main.py`	Backend Engineer	Platform Architect	single-point backend control	Low
`src/model/`	AI Engineer	ML Engineer	inference/runtime coupling	Low
`src/training/`	ML Engineer	AI Engineer	training/export mismatch	Medium
`src/data/`	ML Engineer	Backend Engineer	data quality and lineage risk	Medium
`infra/`	Platform Architect	SRE	monitoring/ops misconfiguration	Medium
`docs/`	Tech Lead	all roles	drift and stale guidance	Low

23.2 Service ownership matrix

Service	Primary owner	Secondary owner	Risk note
Frontend SPA	Nguyen Le Hong Nhi	Duong Hong Quan	API contract drift impacts UX
FastAPI runtime	Pham Duc Long	Le Quang Tuyen	runtime ownership is concentrated
ML training pipeline	Duong Binh An	Do Quoc Trung	training/export path needs dual expertise
Monitoring stack	Le Quang Tuyen	Pham Duc Long	low observability depth if single owner unavailable

23.3 Runtime ownership matrix

Runtime area	Owner	Dependency	Risk
API startup and health	Backend Engineer	`src/main.py`, `BaselineModelInference`	startup failure blocks whole product
Prediction and explanation	AI Engineer	`src/model/baseline.py`, SHAP, ABSA	expensive code path concentrated in one class
Batch and evaluation	Backend Engineer + ML Engineer	pandas, thread pool, MLflow	process-local state and large IO surface
Monitoring and dashboards	Platform Architect / SRE	Prometheus/Grafana config	weak alerting and default creds

23.4 Model ownership matrix

Model area	Owner	Evidence	Risk
Sentiment inference	ML Engineer	`BaselineModelInference.predict_single()` and `predict_batch()`	backend mode selection can change runtime shape
Sarcasm detection	AI Engineer	`_predict_sarcasm_flag()` and sarcasm adapter paths	shared head semantics are easy to misuse
ABSA extraction	AI Engineer	`_extract_aspects()` + zero-shot pipeline	expensive and failure-prone under load
SHAP explanation	ML Engineer	`get_shap_explanation()`	high latency and memory cost

23.5 Knowledge concentration and operational risk

- **Bus factor**: low for runtime and model work because the center of gravity is `src/main.py` + `src/model/baseline.py`
- **Knowledge concentration**: high around ONNX export, finetuning, and runtime mode switching
- **Operational risk**: a single person failure can strand startup, deployment, or model-debug

workflows

- **Mitigation**: add ownership annotations in docs, reduce `main.py` concentration, and add runbooks for startup, predict, explain, batch, and export

24 Section 24 - Critical File Analysis

24.1 Top 20 critical files

File	Purpose	Runtime role	Dependencies	Called by	Impact	Score
`src/main.py`	API entrypoint	critical serving path	FastAPI, contracts, model, metrics	frontend, users, tests	outage blocks system	10
`src/model/baseline.py`	model orchestration	critical inference core	torch, transformers, onnx, shap, peft	`src/main.py`	wrong model behavior	10
`contracts/schemas.py`	HTTP contract	request/response validation	pydantic	main, tests, frontend types	contract drift	9
`contracts/model_interface.py`	model abstraction	model boundary	dataclasses	baseline, mock, tests	coupling risk	9
`src/model/onnx_inference.py`	ONNX runtime bridge	fast inference path	onnxruntime, tokenizer	baseline	runtime failure	9
`src/model/onnx_export.py`	ONNX export	artifact production	optimum, peft, transformers	export script, DVC	deploy artifact failure	9
`src/scripts/run_training.py`	training CLI	offline training	datasets, transformers, peft, mlflow	DVC, trainer container	no adapter produced	9
`src/data/pipeline.py`	preprocessing	raw->processed transform	pandas, transforms, params	DVC	broken dataset lineage	8
`src/data/validation.py`	quality gate	validation step	pandas, json, mlflow URI helpers	DVC validate	bad data acceptance	8
`docker-compose.yml`	runtime topology	deploy wiring	containers, env vars, volumes	operators	service miswire	9
`Dockerfile`	backend image	runtime packaging	DVC, secrets, offline model downloads	build pipeline	bad image or missing artifacts	9
`Dockerfile.train`	training image	trainer packaging	torch, requirements, source copy	training workflow	training environment failure	8
`dvc.yaml`	pipeline graph	offline orchestration	raw/processed/models/reports paths	DVC	broken lifecycle	9

File	Purpose	Runtime role	Dependencies	Called by	Impact	Score
params.yaml	configuration source	shared runtime+training params	yaml consumers	all workflows	config drift	9
src/monitoring/metrics.py	request/inference metrics	prometheus-client	FastAPI middleware	blind spots	8
app/.../sentiment-analysis.service.ts	frontend API integration	HttpClient, signal state, localStorage	UI components	UX breakage	8
app/.../app.component.ts	frontend shell	startup and user interaction	service, UI state, health polling	browser	UX lockout	8
src/training/task_configs.py	task-specific registry	dataclass	training CLI	wrong task setup	8
src/training/dataset_builder.py	dataset building	dedup / stratify / oversample	pandas	finetuning CLI	skewed training set	8
src/model/config.py	runtime configuration	artifact/model path selector	dataclass	baseline, exporter, runtime	wrong mode/path	8

24.2 Top 50 critical files

The following ranked continuation remains high-value because these files participate in test coverage, helper utilities, training behavior, and deployment support:

- src/training/weighted_trainer.py, src/training/class_weights.py, src/training/lora_config.py, src/training/metrics.py, src/training/mlflow_callback.py
- src/data/downloader.py, src/data/utils.py, src/data/transforms/*
- src/model/evaluate.py, src/model/language_detector.py, src/model/device.py, src/model/download_models.py
- src/scripts/prepare_eval.py, src/scripts/evaluate_finetuned.py, src/scripts/export_onnx.py, src/scripts/benchmark_onnx.py, src/scripts/generate_shap_plots.py
- infra/prometheus/prometheus.yml, infra/prometheus/alert_rules.yml, infra/grafana/provisioning/*
- tests/* because they encode the intended contract, runtime behavior, and failure assumptions

24.3 Top 100 critical files

The full top-100 inventory is the union of the top-20 files above and the broader supporting set under `src/training/`, `src/data/transforms/`, `src/scripts/`, `tests/`, `infra/`, and the app sub-tree under `app/sentiment-analysis-chatbot/src/app/`. The ranking criterion is simple: files are critical when changing them can alter a production request path, a training artifact, a deployment artifact, or the contract between those planes.

25 Section 25 - API Analysis

25.1 Endpoint matrix

Endpoint	Purpose	Schema	Validation / deps	Failure modes	Sync	Risk
`/health`	readiness check	`HealthResponse`	`get_model()`, model loaded	503 when model absent	sync	low
`/predict`	single sentiment	`PredictRequest` / `PredictResponse`	text length, language guard, model inference	400 unsupported language, 500 model failure	sync	high
`/explain`	SHAP explanation	`ExplainRequest` / `ExplainResponse`	same as `/predict` plus SHAP runtime	slow/failed explanation	sync	high
`/batch_predict`	CSV batch scoring	multipart file -> `BatchPredictResponse`	CSV parse, `text` column, 500-row cap	invalid CSV, empty rows, batch runtime error	async wrapper over thread	high
`/batch_status/{job_id}`	job status	`BatchStatusResponse`	in-memory job registry	unknown job_id, hardcoded payload semantics mismatch	sync	medium
`/evaluate`	background evaluation trigger	none / `BackgroundTasks`	singleton in-memory guard	409 busy, runtime error	async spawn	medium
`/metrics`	Prometheus scrape	plain text metrics	prometheus client exposition	scrape blank/unavailable if app down	sync	medium

25.2 Execution flow and risk by endpoint

- `/health`: depends on `get_model()` and `BaselineModelInference.is_loaded`; main risk is false healthy if supporting sub-models fail after startup.
- `/predict`: resolves language via `resolve_request_language()` then calls `predict_single()`; performance risk is ABSA and sarcasm cost inside a user-facing request.
- `/explain`: calls `get_shap_explanation()` and is the most expensive synchronous route; security risk is resource exhaustion.

- `/batch_predict`: intentionally skips ABSA to preserve throughput; it is asynchronously wrapped but still CPU-bound inside a thread.
- `/batch_status/{job_id}`: mocked endpoint; it should be treated as non-authoritative.
- `/evaluate`: background trigger with process-local state only; useful for dev but not durable orchestration.
- `/metrics`: operational visibility endpoint; exposure should be network-restricted.

26 Section 26 - Execution Flow Analysis

26.1 Startup

```
flowchart TD
  U[uvicorn] --> L[lifespan()]
  L --> C[ModelConfig(mode from env)]
  C --> B[BaselineModelInference()]
  B --> M[_load_model()]
  M --> P[preload()]
  P --> H[/health becomes ready/]
```

26.2 Predict

```
flowchart TD
  R[POST /predict] --> V[PredictRequest validation]
  V --> G[get_model()]
  G --> L[resolve_request_language()]
  L --> S[BaselineModelInference.predict_single()]
  S --> A[_extract_aspects()]
  S --> T[_predict_sarcasm_flag()]
  S --> O[PredictResponse]
```

26.3 Explain

```
flowchart TD
  R[POST /explain] --> V[ExplainRequest validation]
  V --> G[get_model()]
  G --> L[resolve_request_language()]
  L --> S[BaselineModelInference.get_shap_explanation()]
  S --> X[shap.Explainer]
  X --> O[ExplainResponse]
```

26.4 Batch predict

```
flowchart TD
  R[POST /batch_predict] --> F[file read]
  F --> P[pandas.read_csv()]
  P --> C[text column / 500 row checks]
```

```
C --> T[asyncio.to_thread(_run_batch)]
T --> B[model.predict_batch(skip_absa=True)]
B --> R2[BatchPredictResponse]
```

26.5 Evaluation, training, export, deployment

Evaluation is triggered by `/evaluate` and implements a dev-style background run rather than a persistent job service. Training starts at `src/scripts/run_finetuning.py::main()` and follows `train()` into dataset shaping, PEFT construction, trainer execution, and adapter save. Export starts at `src/scripts/export_onnx.py::main()` and uses `OnnxExporter.export_fp32()` then `export_int8()`. Deployment is the composition of `Dockerfile`, `Dockerfile.train`, and `docker-compose.yml`; the runtime contract is "backend container owns model load, frontend proxies `/api/*`, monitoring sidecars expose visibility".

27 Section 27 - AI Inference Analysis

27.1 Stage analysis

Stage	Input	Transformation	Output	Latency	Memory	Failure
Language detection	text + optional lang	`LanguageDetector` or request override	lang/confidence	low	low	unsupported language
Tokenizer	text(s)	HF tokenization + padding/truncation	tensors / np arrays	medium	medium	tokenizer/load failure
Sentiment classification	tensors	ONNX or HF forward pass + softmax	sentiment + confidence	low to medium	medium	model failure
Sarcasm detection	text	adapter or ONNX sarcasm head	sarcasm flag	medium	medium	head mismatch, load failure
ABSA	text	zero-shot aspect and sentiment passes	aspect list	high	medium to high	pipeline load/runtime failure
SHAP	text	explainer over classification pipeline	token attributions	very high	high	explanation failure

27.2 Provider selection and quantization

`OnnxInferenceSession` selects `CPUExecutionProvider` by default and conditionally prefers CUDA or CoreML when the device and runtime providers are available. `OnnxExporter.export_int8()` uses `AutoQuantizationConfig.avx512_vnni(is_static=False, per_channel=True)` which is a CPU-oriented optimization and therefore only pays off when the deployment host matches

the target instruction set. The operational implication is that inference performance is deployment-hardware-sensitive and should be benchmarked on the target machine rather than assumed from export settings.

27.3 Model loading

`BaselineModelInference._load_model()` is the central runtime switch. In `onnx` mode it loads one or two ONNX sessions; in `finetuned` mode it loads the base model plus sentiment and sarcasm adapters; otherwise it loads the baseline HF model. This is the core architectural decision that ties runtime behavior to `ModelConfig.mode`.

28 Section 28 - ML Pipeline Analysis

28.1 Artifact lineage

Artifact	Producer	Consumer	Format	Lifecycle
Raw sentence/aspect CSVs	`src/data/downloader.py`	`src/data/pipeline.py`	CSV	source-of-truth input
Processed CSVs	`src/data/pipeline.py`	training, validation, evaluation	CSV	intermediate tracked artifact
Quality report	`src/data/validators.py`	human review, DVC metrics	JSON	validation gate output
Adapters	`src/scripts/run_finetuning.py`	runtime `finetuned` mode	PEFT adapter folder	deployable learned artifact
ONNX FP32/INT8	`src/model/onnx_export.py`	runtime `onnx`/`onnx_int8` mode	ONNX model dir	serving artifact
Benchmark report	`src/scripts/benchmark_onnx.py`	manual review	JSON	performance evidence
MLflow run	trainer/eval scripts	experiment tracking	MLflow artifacts	reproducibility record

28.2 Pipeline stages

- **Raw data**: `src/data/downloader.py` parses or downloads the source datasets.
- **Preprocessing**: `src/data/pipeline.py` composes `LabelMapper`, `SentimentDeriver`, `TextCleaner`, `DuplicateRemover`, `LengthFilter`, and `Splitter`.
- **Validation**: `src/data/validators.py` checks shape, labels, null ratios, orphans, and distribution warnings.

- **Training**: `src/scripts/run_finetuning.py` builds PEFT tasks from `src/training/task_configs.py`.
- **Evaluation**: `src/scripts/evaluate_finetuned.py` and `src/model/evaluate.py` produce metrics and fairness reports.
- **Export**: `src/scripts/export_onnx.py` and `src/model/onnx_exporter.py` transform adapters into deployable ONNX.
- **Benchmarking**: `src/scripts/benchmark_onnx.py` measures the runtime path.

29 Section 29 - Security Analysis

29.1 Threat model

Threat	Evidence	Severity	Likelihood	Mitigation
Unauthenticated access	all routes in `src/main.py` are public	High	High	add authN/authZ and per-route policy
Wildcard CORS with credentials	`allow_origins=["*"]` and `allow_credentials=True`	High	High	restrict origins and credential scope
Default Grafana admin password	`GF_SECURITY_ADMIN_PASSWORD=admin` in compose	High	High	secrets management and rotated password
Sensitive build arguments	`DAGSHUB_USERNAME/DAGSHUB_TOKEN` in Docker build args	High	Medium	use secret injection, not build args
File upload abuse	`/batch_predict` accepts multipart CSV uploads	Medium	Medium	enforce size, content, and rate limits
Artifact trust	runtime depends on local files and DVC pull results	Medium	Medium	add integrity verification and pinned provenance
Dependency risk	many ML packages with rapid release cadence	Medium	Medium	pin versions, scan and stage upgrades

29.2 Trust boundaries

The critical trust boundary is between the browser and the API, because the frontend does not protect the backend and the backend currently accepts direct public requests. The second boundary is between the API container and the artifact volumes, because serving depends on locally mounted model and data files. The third boundary is between build-time credentials and runtime artifacts, because DVC pull behavior is part of the image build.

30 Section 30 - Performance Analysis

30.1 Workload model

Workload	Primary code path	First bottleneck	Second bottleneck	Long-term bottleneck	Notes
100 req/min	`predict_single()`	ABSA/SHAP when enabled	model load contention	single backend process	feasible if auxiliary features are limited
1,000 req/min	`predict_batch()` and ONNX	CPU saturation	Python orchestration	memory pressure	needs batching and mode discipline
10,000 req/min	frontend/API ingress	request queuing	model process scaling	shared artifact IO	single process is insufficient
100,000 req/day	mixed	startup and capacity	explanation cost	operational churn	needs durable job model and autoscaling

30.2 Latency and memory analysis

Startup time is dominated by `BaselineModelInference._load_model()` and `preload()`, especially in finetuned and ONNX modes. Inference latency is lowest when `skip_absa=True` and SHAP is avoided. ABSA latency is high because each detected aspect triggers another zero-shot forward pass. SHAP latency is high because the explainer constructs a local surrogate explanation over the classification pipeline. Memory pressure rises with model size, tokenizer objects, SHAP explainer state, and concurrent batch requests.

31 Section 31 - Observability Analysis

31.1 Coverage

- **Metrics**: implemented through `src/monitoring/metrics.py`
- **Monitoring**: implemented through Prometheus and Grafana provisioning
- **Logging**: partially implemented; logs exist, but no structured centralized log strategy is present
- **Tracing**: not implemented
- **Alerting**: partially implemented through alert rules, but no evidence of comprehensive incident routing
- **SLO/SLI**: not explicitly implemented

31.2 Blind spots

The major blind spots are the absence of trace IDs, the lack of request-level correlation across frontend/backend/trainer, and the lack of durable incident state for evaluation and batch workflows. This reduces production diagnosability even though basic metrics exist.

32 Section 32 - Failure Mode Analysis

32.1 Fault tree summary

Workflow	Symptoms	Root causes	Recovery path	Impact
Startup	API never becomes healthy	artifact missing, wrong mode, load failure	fix config/artifacts and restart	total outage
Prediction	400/500 from API	unsupported language, inference failure, bad schema	validate input and inspect model mode	user-visible failure
Explanation	slow or failing explain call	SHAP/tokenizer/runtime failure	retry with model limits or disable explain	degraded UX
Batch processing	upload fails or partial rows	CSV parse, missing column, batch runtime error	fix input, reduce size, inspect logs	blocked batch workflow
Training	no adapter output	raw data missing, trainer failure	rerun upstream stages	no learning artifact
Export	no ONNX output	merge/export/quantization failure	verify adapter and rerun export	no deployable model
Deployment	stack starts partially	DVC pull/build/env failure	rebuild with valid artifacts	service unavailability
Monitoring	dashboards empty	scrape/provisioning drift	verify metrics endpoint and configs	poor incident detection